

uc3m

Universidad
Carlos III
de Madrid

Máster en Ingeniería Informática

Datos Masivos y Encadenados 2024-2025
Grupo 1

Práctica Obligatoria

“Parte 1: Integración de datos”

Luis Daniel Casais Mezquida – 100429021

Lucas Gallego Bravo – 100429005

Till Niklas Köbele – 100548395

Francisco Montañés de Lucas – 100406009

Diego Picazo García – 100549459

Equipo 3

Profesor

José Luis López Cuadrado

Índice

1. Resumen ejecutivo	2
2. Prueba de concepto	3
2.1. Alcance	3
2.2. Arquitectura implementada	3
2.3. Implementación	4
2.4. Ejecución	11
2.5. Escalabilidad	11
3. Conclusiones	12
4. Referencias	13

1. Resumen ejecutivo

Lo que se quiere conseguir a la hora de realizar este trabajo, es conseguir reunir en una única página web los lugares del planeta en los que se han conseguido certificar o validar avistamientos de ovnis. Mediante la utilización de una amplia gama de fuentes de datos heterogéneos (CSV, APIs, www...) pertenecientes a diferentes satélites y empresas se quiere lograr trazar en un mapa global en tres dimensiones dichos datos de tal forma que se pueda ver la ubicación del avistamiento y los datos relevantes a ese suceso. Estas amplias fuentes de datos cuentan con datos en diferentes formatos, por lo que se cuenta con datos auditivos, imágenes, video y demás. Además, se quiere utilizar los datos pertenecientes a distintas compañías, tanto públicas como privadas, con el fin de contrastar la información obtenida mediante diferentes fuentes sobre cada avistamiento. Además, gracias al estado actual de la inteligencia artificial, se puede hacer uso de la misma con el fin de agilizar el trabajo, ya que por ejemplo, si se cuenta con audios muy largos, se puede utilizar esta tecnología para que seleccione las secciones más interesantes de ese audio.

Con el apoyo de una red interna Docker, Rust para el componente principal del *backend* y Python como experto analista y extractor de datos, se busca alimentar una base de datos *Mongo NoSQL* que ofrezca una prueba de concepto visual de la mano de React, de lo que puede llegar a ser una amplia red de avistamientos y fenómenos astronómicos que despierten el interés de aficionados y de la comunidad científica con el afán de seguir explorando el espacio.

2. Prueba de concepto

Para la prueba de concepto, se ha decidido realizar una versión simplificada de la arquitectura general, a fin de mostrar el potencial de la aplicación final.

2.1. Alcance

Esta versión reducida incorpora un prototipo de la interfaz de la aplicación, mostrando sobre un mapa geográfico los distintos eventos disponibles en la base de datos. Se siguen mostrando los detalles de cada elemento, así como una lista de elementos generales, pero no se permite la realización de filtros ni consultas.

Se ha decidido incluir como fuentes de datos una página web sobre la que realizar *scraping*, una base de datos estática, y una API de terceros, a fin de mostrar los tres tipos de extracción y agregación de datos.

También se han reducido los tipos de datos recogidos a dos: avistamientos y eventos astronómicos, dado que la agregación de datos de satélites requiere en varios casos de un pago por consulta, y de la utilización de modelos de IA para el análisis de esos datos.

Para simplificar el desarrollo de los extractores de datos, el *backend* es el encargado de guardar en las bases de datos los contenidos, en lugar de que lo haga cada agregador por separado.

Por último, se ha decidido realizar un servicio específico para la extracción de datos de APIs a terceros, el cual se guarda en base de datos, para así reducir el número de consultas a las API.

El resultado es una aplicación funcional que muestra la idea en funcionamiento, y el potencial que tiene.

2.2. Arquitectura implementada

La arquitectura resultante queda plasmada por la Figura 1:

Esta arquitectura es una versión reducida y simplificada de la arquitectura final. Cuenta con los mismos elementos básicos, y simplifica el camino por el que deben pasar los datos. Como hemos mencionado en el apartado anterior, los extractores de datos almacenan los datos a través del *backend*. Se cuenta con un único *data warehouse*, con separación entre tipos de datos. El *frontend* consume directamente del *backend* los datos a mostrar por pantalla, los cuales han de ser recargados manualmente.

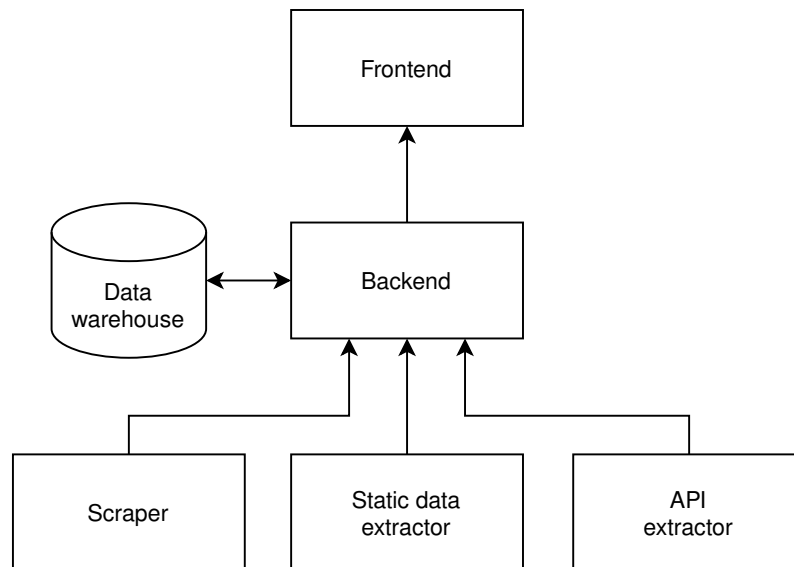


Fig. 1: Arquitectura de la prueba de concepto

2.3. Implementación

Para la implementación se ha optado por el uso de contenedores Docker [1], uno por elemento de la arquitectura, para facilitar tanto el desarrollo como el despliegue de la aplicación, ya que cada elemento es independiente del resto, permitiendo el desarrollo en distintos lenguajes de programación.

2.3.1. Backend

El backend está implementado en el lenguaje de programación Rust, haciendo uso del *framework* Rocket [2]. Consta de un servidor web con una API tipo REST, la cual se basa en el protocolo HTTP. Para la integración con la base de datos de MongoDB, se usó la librería de Rust que proporciona el propio Mongo.

El servicio cuenta con dos rutas para la interacción con los dos tipos de datos implementados: `/sightings` y `/events`. Cada una de estas rutas permite tres tipos de acciones:

- Petición GET a `/<tipo>`: Retorna una lista de todos los elementos almacenados en base de datos correspondientes a ese tipo, realizando la correspondiente consulta.
- Petición GET a `/<tipo>/<id>`: Retorna los detalles del elemento requerido.
- Petición POST a `/<tipo>`: Almacena en base de datos el elemento especificado, siguiendo el esquema definido en la Subsección ??.

En caso de que al devolver los elementos, éstos no cuenten con información sobre la localización geográfica precisa del elemento, realiza una llamada a la API de Mapbox [3] para obtenerla.

2.3.2. Data warehouse

Para el almacenamiento de datos, se optó por usar MongoDB [4], dada su amplio uso en el campo de *big data*, su facilidad para almacenar grandes cantidades de datos de manera distribuída, y el hecho de que es una base de datos no relacional.

Se creó una única base de datos (*jose*) con dos colecciones: una para los avistamientos (*sightings*) y otra para los eventos astronómicos (*astronomical-events*).

Se decidió usar la versión de contenedor de Docker dado que permite una mayor flexibilidad en el despliegue, al mismo tiempo que está empaquetada con el resto de la aplicación (para más fácil distribución) y no requiere de instalación en el sistema. También se habilitó la persistencia de datos mediante un volumen de Docker, para así mantener los datos ya extraídos aunque se reinicie la aplicación.

2.3.3. Frontend

Para el frontend nuestra prueba de concepto implementa todas las funciones fundamentales:

- Accede a los datos recompilados según el backend
- Muestra los elementos en el mapa (donde aplicable)
- Tiene listas de todos los elementos disponible (ordenados por proximidad donde aplicable) y una opción de manualmente recargar los datos del backend.
- Ofrece una vista de detalles para elementos específicos

Biblioteca JavaScript Usamos *React* cómo herramienta básica.

Nuestros requisitos funcionales para el frontend forman un ejemplo destacado de un *Single Page Application* (aplicación de una sola página), porque por diseño es una sola página con comportamiento complejo. Por lo tanto, React es una opción obvia. La implementación también beneficio de su gran foco a componentes individuales.

Marco de interfaz de usuario Para estructurar la interfaz usamos *Chakra-UI*.

Ayuda con acelerar el desarrollo con bloques de construcción predefinidos. En la actualidad, no es personalizado mucho, pero ofrece opciones correspondientes si es necesario en el futuro. Ya que la estructura básica aún no es especialmente complicada, también puede ser retirado o sustituido fácilmente.

Mapa El núcleo del frontend constituye la integración con *Mapbox*.

La gran ventaja de usar Mapbox sobre otro proveedor de mapas es que el uso se mide por las cargas de página y no baldosas solicitado (que pase siempre que hay movimiento). Ya que solo usamos un mapa, pero movemos mucho, en nuestro caso es ideal.

Usamos la API de Mapbox en directo con interacciones del usuario, directamente sobre el paquete JS proporcionada por el promotor. Los elementos se pasan cómo una capa con símbolos propios al mapa. Todas las interacciones con las partes React de la página web son integradas. Por ejemplo se mueve a la ubicación de un elemento sobre la selección.

2.3.4. Scraping

Para el apartado de implementación de *web scraping* se ha creado un fichero de código empleado el lenguaje de programación **python**. Para esta tarea se ha utilizado la librería de *web scraping* denominada *Playwright* con la cual se han podido obtener diferentes datos de interés para nuestro proyecto pertenecientes a la página web UFO Stalker, la cual cuenta con una amplia base de datos la cual contiene información sobre posibles avistamientos en diferentes partes del mundo. Estos datos son subidos por los usuarios o personas que han presenciado dicho avistamiento, por lo que en algunos casos puede tratarse de falsos positivos.

Otras librerías necesarias son *Requests* para poder mandar los datos obtenidos mediante *web scraping* a la base de datos. La librería *Json*, ya que se ha creado un fichero de esta extensión con el cual se comprueba que no se obtengan datos de páginas ya visitadas anteriormente con el fin de evitar duplicados. Otra librería es *Time*, la cual se utiliza para añadir *sleeps* dentro del código para evitar sobrecargar la página de la que se obtiene información. También se utiliza *Datetime* para modificar el formato los tiempos que se obtienen de la página web y transformarlos a formato *Unix Timestamp* -s. Finalmente, se utilizan las librerías *Re* y *Logging*, la primera de ellas se utiliza para modificar algunos de los datos que se obtienen eliminando cualquier elemento del string obtenido que no sea un dígito, esto se hace ya que en algunos casos, en campos que deben ser enteros aparecen cosas como *Roughly 1 metter*", de esta forma obtenemos el 1 eliminado así el resto de elementos de la string. La otra librería se utiliza para usar *loggings* en vez de *prints* en el código.

Para la correcta obtención de los datos se ha tenido que estudiar el HTML de dos páginas web, la primera de ellas es una tabla en la que cada fila es un avistamiento diferente, y al hacer click en cualquiera de las filas se redirige al usuario a la propia página del avistamiento que es donde se encuentran todos los datos, este análisis de los HTML se ha realizado utilizando las herramientas de desarrolladores que ofrece el navegador web Google Chrome para obtener así las URLs necesarias para posteriormente poder ir a la página de ese avistamiento y obtener los nombres de los contenedores en los cuales se encuentran los diferentes datos del avistamiento producido que queremos obtener. Una vez analizados los HTML y obtenidas las URLs y los nombres de los contenedores, se

puede pasar a escribir el código necesario para la realización del *web scraping*.

A continuación se adjuntan unas imágenes de las páginas web para mostrar cómo es la tabla previamente mencionada y como se ven los datos una vez haces click en una de las filas de la tabla. Cabe destacar que la tabla tiene como máximo 10 filas, por lo que una vez se han leído esas 10 filas se debe pasar a una siguiente página:

UFO STALKER

Report a UFO | Sighting Search | Sighting List | Sighting Map

Show 10 Sightings

Occurred	Location	Description	
12/12/2024 4:32 AM	Vancouver, British Columbia	While kayaking in False Creek, a giant metallic jellyfish-like UFO appeared, hovering and emitting rhythmic vibrations before zigzagging into the clouds.	
12/10/2024 2:02 PM	Santa Fe, New Mexico	While trying to get my telescope to work, I noticed an oval-shaped object hovering, changing colors from metallic silver to aqua blue.	
12/13/2023 1:00 AM	North Andover, Massachusetts	While driving home around midnight with friends, I witnessed a green light in the sky that swiftly flew upward and vanished faster than anything I've ever seen.	
12/09/2024 4:21 AM	Austin, Texas	While enjoying a jazz club in Austin, a shimmering disc with an iridescent glow appeared outside, pulsing and rotating above a nearby rooftop. The club's patrons were captivated as it hovered silently, revealing indistinct shapes inside.	
12/12/2024 1:02 PM	Winnipeg, Manitoba	While tending to my botanical garden, a metallic disc hovered overhead, casting a glowing light that seemed to animate the flowers.	
12/09/2024 9:16 AM	Tucson, Arizona	A shiny, hubcap-like object with a blue glow hovered above the backyard garden, emitting an eerie static sound that interrupted the radio.	
12/11/2024 5:11 AM	Durango, Durango	While observing the stars with a telescope, a massive oval-shaped object appeared above the mountains, glowing with an iridescent light. It hovered silently, emitted scanning beams, and then sped away at an impossible speed.	
12/11/2024 11:00 PM	Auvergne-Rhône-Alpes	A triangular ship emits a green light that causes cats to disappear, suggesting it is responsible for their abductions.	
12/09/2024 9:08 AM	Portland, Oregon	While fishing on the Willamette River, a shimmering, jellyfish-like object zipped across the sky, causing ripples in the water and making fish jump. It hovered and emitted warmth.	

Fig. 2: Captura de la tabla Ufo Stalker.

UFO STALKER

Report a UFO | Sighting Search | Sighting List | Sighting Map

Sponsored
Your Ad Here
[Contact us to advertise here](#)

UFO Sighting

Fingerprint

FMLYLUP1

Summary

While kayaking in False Creek, a giant metallic jellyfish-like UFO appeared, hovering and emitting rhythmic vibrations before zigzagging into the clouds.

Submitted

12/14/2024 2:02 PM

Occurred

12/12/2024 4:32 AM

Variance

Reported 2 Days Later

Accuracy

Approximate

Location

Vancouver, British Columbia

Detailed Description

So I'm chilling in my kayak on the still waters of False Creek, you know, just trying to escape the city noise. The sun's dipping below the horizon, painting the sky in this insane orange-pink mix. Suddenly, this weird buzzing noise fills the air. I look up and see this thing, like a giant metallic jellyfish, floating silently above the water. It was like nothing I've ever seen before, with shimmering colors pulsating across its surface. It just hovered there, making these strange, rhythmic vibrations, and then, as if it had spotted me, it shot upwards in a zigzag pattern before disappearing into the clouds.

Additional Details

Rating

☆☆☆☆☆

Type

UFO Sighting

Source

UFO Stalker

Labels

Routing

Shape

Jellyfish

Features

Metallic, Pulsating Colors, Rhythmic Vibrations

Flight Path

Hovering, Zigzag, Vertical Ascension

Distance

200 meters

Duration

3 Minutes

Altitude

500 feet

Fig. 3: Captura de un avistamiento en Ufo Stalker.

Como ya se ha mencionado para poder realizar el *web scraping* se ha creado un archivo de código empleando el lenguaje de programación **python**. Este archivo cuenta con un

total de dos funciones además de un main, y obtiene actualmente un total de 15 datos de avistamientos cada dos minutos y medio.

La primera de estas funciones recibe el nombre *get_urls()*, y es la encargada de recorrer las filas de la tabla previamente mencionada y obtener las URLs de cada avistamiento. Para esto, esta función recibe del main dos parámetros, el primero de estos corresponde con el número de urls que deben obtenerse por cada iteración, en este caso quince, el siguiente parámetro indica a la función de que URL debe obtener los datos. Posteriormente, se abrirá el archivo json en el cual se guardan todas las url ya visitadas con el fin de evitar generar duplicados, y se creará un nuevo browser de chromium y una página de ese browser para poder navegar con *Playwright*. Una vez creado el browser y cargada la página indicada por la url, entraremos en un bucle de recopilación de urls hasta llegar el número indicado como parámetro, primero mediante un *try* se tratara de localizar la ventana emergente que pide consentimiento de cookies, si se localiza se hará click en el botón de consentimiento para aceptarlas y poder navegar por la página. Esta ventana emergente solo aparecerá la primera vez que se carga la página, por lo que en las demás ocasiones no generará problemas. Una vez dado el consentimiento, cerrándose así la ventana emergente, entraremos dentro del *while* a un bucle *for*, el cual recorrerá todas las filas de la tabla obteniendo las URLs de cada avistamiento, por cada URL de cada fila que consiga se comprobará si existe ya una entrada con esa URL en el json, de ser así esa URL no se añadirá y el contador no avanzará, si por el contrario no existe, se añadirá tanto al json como a la lista de URLs encontradas. Una vez se han obtenido todas las URLs de las filas visibles se saldrá del bucle *for*, y dentro del *while*, se le dará al botón de siguiente página, y se comprobará si se han obtenido todas las URLs necesarias antes de entrar de nuevo al bucle *for* de la siguiente tabla. Finalmente, se retorna una lista con todas las URLs nuevas obtenidas.

La segunda de función recibe el nombre *get_data()*, la cual recibe un único parámetro, en este caso la lista de URLs obtenidas por la función anterior. Esta función como su nombre indica es la encargada de obtener la información relevante de cada una de las URLs en la lista y mandarlas al *backend* para que se puedan añadir a la base de datos. Al igual que en la función anterior, lo primero que se hace es crear un browser de chromium y una página de ese mismo browser para poder navegar a cada URL. Una vez creados, mediante un bucle *for* se recorrerán las URLs que están dentro de la lista, obteniendo por cada URL los datos relevantes para nuestro proyecto. Una vez obtenidos los datos, se pondrán en el formato requerido por el *backend* y se enviarán mediante un *Post*.

Finalmente, el main de este archivo es el encargado de llamar a las dos funciones previamente comentadas, este main cuenta con un bucle *while True* y un *sleep* de dos minutos y medio para estar constantemente preguntando por quince nuevos datos cada dos minutos y medio, de esta forma no sobrecargamos la página de la que obtenemos información. Se han utilizado otros *sleep* en cada una de las funciones para no realizar las operaciones de click y volver atras de una forma muy rápida, simulando un poco lo que podría llegar a ser una navegación normal.

2.3.5. ETL de datos estáticos

Para la implementación de *ETL de datos estáticos* se ha creado un fichero con un programa informático empleando el lenguaje de programación **Python** y archivos CSV.

Para esta tarea se ha utilizado principalmente la librería de *gestión de datos* denominada *Pandas* con la cual se han podido obtener muchísimos datos de interés para nuestro proyecto (eventos astronómicos, comúnmente relacionados con avistamientos de OVNI) recolectados y organizados en bases de datos de la NASA y ESA (las agencias espaciales de EUUU y la Unión Europea), las cuales son coherentes, similares y en constante refresco mediante a mediciones científicas de estas organizaciones.

Estos datos incluyen actualmente varios conjuntos de datos distintos, entre ellos:

- Datos de la ESA sobre asteroides realizando **close approaches** a la Tierra en el pasado o presente (nombre, tiempo del evento). [5]
- Datos de la NASA sobre asteroides realizando **close approaches** a la Tierra en el pasado o presente (nombre, tiempo del evento). [6]
- Datos de la NASA sobre **bolas de fuego** avistadas sobre la Tierra en el pasado (tiempo del evento, localización). [7]
- Datos de la NASA sobre **meteoritos** que hayan impactado contra la Tierra en el pasado (nombre, tiempo del impacto, localización, composición). [8]

Otras librerías necesarias son *Requests* para poder insertar los datos obtenidos mediante *ETL de datos estáticos* a la base de datos. La librería *JSON* ha sido de gran importancia también, ya que se requiere usar este formato para insertar datos en la base de datos. Otra librería es *Time*, la cual se utiliza para añadir breves *sleeps* dentro del código para evitar sobrecargar la base de datos con peticiones. También se utiliza *Datetime* para adaptar el formato los tiempos que se obtienen de los datos estáticos y transformarlos a formato *Unix Timestamp* requerido por la base de datos. Finalmente, se utiliza la librería *Logging*, para registrar errores.

Para la correcta obtención de los datos se ha tenido que descargar el CSV de las páginas web de la NASA y ESA. A continuación, se han de extraer y corregir los datos, reestructurándolos a la estructura de objetos esperados por nuestra base de datos.

Como ya se ha mencionado para poder realizar el *ETL de datos estáticos* se ha creado un archivo de código empleando el lenguaje de programación **Python**. Este archivo cuenta con cuatro funciones principales para convertir cada uno de los 4 archivos CSV, y obtienen alrededor de 3000-4000 datos, aunque se han excluido unos 40.000 datos de meteoritos para aligerar la carga en el frontend. Aparte de esas funciones principales, se incluyen una gran cantidad de funciones auxiliares para convertir fechas y otros datos desde los formatos originales al esperado por la base de datos.

Finalmente, una vez extraídos y convertidos al formato requerido por el *backend*, se emplea un bucle *for* para recorrer todos los eventos del JSON, y se enviarán al backend mediante una request de tipo *Post*.

2.3.6. APIs externas

Para la implementación de *APIs externas* se ha creado un fichero con un programa informático empleando el lenguaje de programación **Python**.

Para esta tarea se ha utilizado principalmente la librería de *gestión de datos* denominada *JSON* que nos ha servido para convertir el archivo de formato JSON recibido de la llamada API a el JSON en el formato esperado por la base de datos.

De esta forma extraemos datos de interés para nuestro proyecto (eventos astronómicos, comúnmente relacionados con avistamientos de OVNI) recolectados y organizados en una base de datos de la NASA (la agencia espacial de Estados Unidos) de forma coherente y en constante refresco mediante a mediciones científicas de estas organizaciones.

La llamada API nos permite seleccionar de forma precisa los datos que deseamos, filtrando acorde a una API que permite un ajuste muy preciso. Los datos incluyen actualmente un sólo conjunto de datos distintos, importados desde <https://ssd-api.jpl.nasa.gov/cad.api?date-min=2025-01-01&date-max=2026-01-01&dist-max=0.01>

- Datos de la NASA sobre asteroides realizando **close approaches** más cercanos a 0,01 AU a la Tierra en 2025 (nombre, tiempo del evento)

Otras librerías necesarias son *Requests* para poder mandar los datos obtenidos mediante *APIs externas* a la base de datos. Otra librería es *Time*, la cual se utiliza para añadir un brve *sleep* dentro del código para evitar sobrecargar la base de datos con peticiones. También se utiliza *Datetime* para adaptar el formato los tiempos que se obtienen de los datos estáticos y transformarlos a formato *Unix Timestamp* requerido por la base de datos. Finalmente, se utiliza la librería *Logging*, para registrar errores.

Para la correcta obtención de los datos se ha elaborado una petición de API de forma manual. A continuación, se han de extraer y corregir los datos, reestructurandolos a la estructura de objetos esperados por nuestra base de datos.

Como ya se ha mencionado para poder realizar el *APIs externas* se ha creado un archivo de código empleando el lenguaje de programación **Python**. Este archivo cuenta con una función principal para realizar la única llamada API y convertir los datos recibidos del JSON al formato deseado, obteniendo alrededor de una docena de datos por la petición tan precisa realizada (*Close Approaches* inferiores al 1 % de la distancia al sol, en el periodo de 1 año). Aparte de esas funciones principales, se incluyen unas pocas funciones auxiliares para convertir fechas y otros datos desde los formatos originales al esperado por la base de datos.

Finalmente, una vez extraídos y convertidos al formato requerido por el *backend*, se emplea un bucle *for* para recorrer todos los eventos del JSON, y se enviarán al backend mediante una request de tipo *Post*.

2.4. Ejecución

Al estar basado en Docker y Docker Compose, para desplegar la aplicación basta con ejecutar el siguiente comando:

```
docker compose up --build
```

El cual levantará la aplicación en el puerto 8080 de la máquina local, es decir, en <http://localhost:8080>.

2.5. Escalabilidad

Al ser contenedores Docker, la arquitectura permite levantar múltiples instancias de cada elemento, al igual que repartir la carga entre ellos. La arquitectura también permite, al ser modular, que cada elemento se ejecute en máquinas distintas. La base de datos también puede cambiarse de un contenedor a un servidor dedicado, simplemente cambiando la URL a la que apunta el *backend*.

Todo esto hace que la aplicación sea robusta, segura, y escalable.

3. Conclusiones

Durante la realización de esta práctica se ha logrado cumplir todos los objetivos iniciales propuestos.

- Desarrollo de un backend y base de datos muy potente usando Rust y Mongo, capaz de almacenar y manejar grandes cantidades de datos a grandes velocidades.
- Desarrollo de importación, conversión y almacenaje de datos mediante web scraping usando Python
- Desarrollo de importación, conversión y almacenaje de datos mediante ETL de datos estáticos (en CSV) usando Python
- Desarrollo de importación, conversión y almacenaje de datos mediante API usando Python
- Desarrollo de un frontend con un mapa interactivo, capaz de extraer datos y mostrarlos de forma efectiva.

Se hallaron numerosos problemas, que fueron resueltos durante el desarrollo de la práctica:

- Dificultades respecto a la definición de la estructura de la base de datos
- Dificultades a la hora de pulir la interfaz del frontend y tratar las llamadas a API de Mapbox
- Dificultades a la hora de scrapear una página web dinámica, la mayoría de recursos para scrapeo de HTML no funcionan con páginas Javascript.
- Dificultades a la hora de encontrar plataformas de API para extraer localizaciones precisas a partir de localizaciones imprecisas.
- Dificultades a la hora de tratar con las conversiones de datos al formato correcto
- Dificultades a la hora de dockerizar el sistema

A lo largo de la práctica hemos aprendido mucho sobre como tratar datos masivos, y consideramos que hemos tratado con éxito los objetivos del curso, superponiendonos a los problemas hallados.

4. Referencias

- [1] «Docker.» (n.a.), dirección: <https://www.docker.com/> (visitado 13-10-2024).
- [2] «Rocket framework.» (n.a.), dirección: <https://rocket.rs/> (visitado 02-11-2024).
- [3] Mapbox. «Mapbox API.» (2015), dirección: <https://docs.mapbox.com/api/overview/> (visitado 20-10-2024).
- [4] «MongoDB.» (n.a.), dirección: <https://www.mongodb.com/> (visitado 01-11-2024).
- [5] ESA. «ESA NEO Database.» (2016), dirección: <https://neo.ssa.esa.int/close-approaches> (visitado 11-11-2024).
- [6] NASA. «NASA CNEOS Database.» (2017), dirección: <https://cneos.jpl.nasa.gov/ca/> (visitado 11-11-2024).
- [7] NASA. «NASA Fireballs-Bolides Database.» (2017), dirección: <https://cneos.jpl.nasa.gov/fireballs/> (visitado 11-11-2024).
- [8] NASA. «NASA Meteorite Database.» (2015), dirección: https://data.nasa.gov/Space-Science/Meteorite-Landings/gh4g-9sfh/about_data (visitado 09-11-2024).